Finishing string matching (find function):

s1	h	e	l	l	o	!	!	!	!	!
s2	l	l	o							

Idea: test for match at all possible starting points: 0, 1, 2, ... n1 − n2

where n1 = s1.length(), n2 = s2.length().

Sketch:

```
for(i = 0; i <= n1 - n2; i++) {
    // check for match @ i
    // s1[i] == s2[0] AND s1[i+1] == s2[1]
    bool match = true;
    for (j = 0; j < n2; j++)
        match &&= (s2[j] == s1[i+j]);
    if (match) return i;
}
return -1; // -1 ≡ string::npos
```

Exercise: counting words in a string

"we are doomed"

count # spaces, subtract 1 ??

Works a lot of the time, but some people place two spaces between sentences...

Can we do something more robust?

_we__are___doomed_

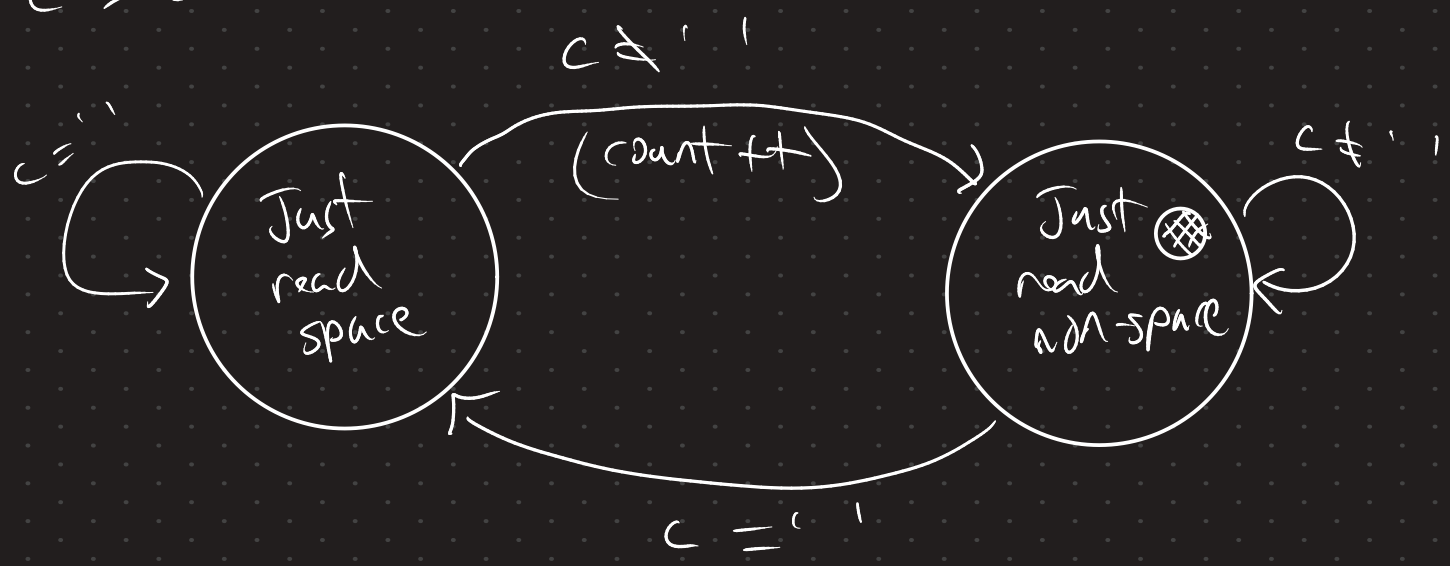
Idea count word boundaries. Lets go with word beginnings.

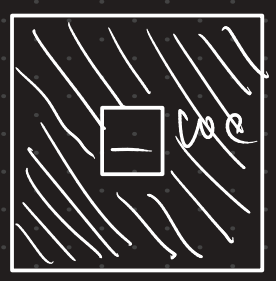
How to find? Need to look for transition from space → non-space.

New Tool: Finite State Machines.

We'll model the following diagram in program code: (see 'lex' & 'yacc')

c ≡ current character





__ are ___ doomed _

How to model in code?

- use an integer to keep track of current state (or bool if #states == 2)
- conditionals for each arrow, for each state.

```
int state = 0;   // 0 is left (just read space)
size_t count = 0;   // 1 is right (non-space)
char c; //current char
while(/* c = next char */) {
    if(state == 0) {
```

```
        if(c != ' ') {
            state = 1; // move token
            count++;
        } // else ...
    } else{ // just read non space
        if(c == ' ') {
            state = 0; // move token back
                       //        to left
        } // else... nothing to do.
    }
    return count;
```